

oskernel2023-avx

04_process_smp — 进程/线程/调度与多核

Q04_001 · short_answer

执行实体 (Execution Entity) 抽象是什么？ 请按以下格式作答（每项必须有代码证据）： - 顶层类型名: (**如 *Process / Task / Thread / TaskControlBlock***) - 结构体路径: - 关键字段 (**至少列 3 个**) : **Context=**, **State=**, **PID=**, **TrapFrame=** - **是否区分 PCB 与 TCB**: (是 / 否 / 待核实)

项目	方法A	方法B
答案	顶层类型名: struct proc (进程) + struct thread (线程) 双层级 结构体路径: kernel/include/proc.h:56 (struct proc), kernel/include/thread.h:20 (struct thread) 关键字段: - Context: struct proc::context (kernel/include/proc.h:88), struct thread::context (kernel/include/thread.h:37) - State: struct proc::state (enum procstate, kernel/include/proc.h:60), struct thread::state (enum threadState, kernel/include/thread.h:26) - PID: struct proc::pid (kernel/include/proc.h:67) - TrapFrame: struct proc::trapframe (kernel/include/proc.h:86), struct thread::trapframe (kernel/include/thread.h:36) 是否区分 PCB 与 TCB: 是	顶层类型名: Process / Thread 结构体路径: kernel/include/proc.h (struct proc), kernel/include/thread.h (struct thread) 关键字段: Context (struct context context), State (enum procstate state / enum threadState state), PID (int pid / int tid), TrapFrame (struct trapframe *trapframe) 是否区分 PCB 与 TCB: 是
证据	[1] kernel/include/proc.h · proc struct proc { enum procstate state; int pid; struct context context; struct trapframe trapframe; thread main_thread; thread thread_queue; ... } [2] kernel/include/thread.h · thread typedef struct thread { enum threadState state; struct proc p; int tid; context context; struct trapframe *trapframe; ... }	[1] kernel/include/proc.h · proc struct proc { ... enum procstate state; ... int pid; ... struct trapframe trapframe; ... struct context context; ... } [2] kernel/include/thread.h · thread typedef struct thread { ... enum threadState state; ... int tid; ... struct trapframe trapframe; context context; ... } thread;

Q04_002 · short_answer

任务/进程的生命周期状态机有哪些状态与流转点？ (Ready/Running/Blocked/Exited 等；需状态枚举/字段证据)

项目	方法A	方法B
答案	进程状态 (enum procstate, kernel/include/proc.h:53): UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE 线程状态 (enum threadState, kernel/include/thread.h:20): t_UNUSED, t_SLEEPING, t_RUNNABLE, t_RUNNING, t_ZOMBIE, t_TIMING 流转点: - RUNNABLE→RUNNING: scheduler() 选择进程 (kernel/proc.c:715-718) - RUNNING→RUNNABLE: yield() 主动让出 (kernel/proc.c:784-792) - RUNNING→SLEEPING: sleep() 阻塞 (kernel/proc.c:818-847) - SLEEPING→RUNNABLE: wakeup() 唤醒 (kernel/proc.c:851-862) - RUNNING→ZOMBIE: exit() 退出 (kernel/proc.c:567-609) - ZOMBIE→UNUSED: freeproc() 回收 (kernel/proc.c:287-334)	进程状态: UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE (kernel/include/proc.h) 线程状态: t_UNUSED, t_SLEEPING, t_RUNNABLE, t_RUNNING, t_ZOMBIE, t_TIMING (kernel/include/thread.h) 流转点: fork 创建 (RUNNABLE), sched 调度 (RUNNING→RUNNABLE/SLEEPING), exit 退出 (ZOMBIE), wait 回收 (UNUSED)
证据	[1] kernel/include/proc.h · procstate enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE }; [2] kernel/include/thread.h · threadState enum threadState { t_UNUSED, t_SLEEPING, t_RUNNABLE, t_RUNNING, t_ZOMBIE, t_TIMING }; [3] kernel/proc.c · scheduler if (p->state == RUNNABLE) { p->state = RUNNING; p->main_thread->state = t_RUNNING; }	[1] kernel/include/proc.h · procstate enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE }; [2] kernel/include/thread.h · threadState enum threadState { t_UNUSED, t_SLEEPING, t_RUNNABLE, t_RUNNING, t_ZOMBIE, t_TIMING, };

Q04_004 · short_answer

上下文切换保存/恢复了哪些寄存器集合？（例如 RISC-V s0-s11；必须引用汇编/结构体证据）

项目	方法A	方法B
答案	保存/恢复的寄存器 (RISC-V callee-saved): ra, sp, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11 证据: kernel/swtch.S:4-27 保存 14 个寄存器到 struct context, kernel/include/context.h:6-22 定义 context 结构包含这些字段	保存/恢复的寄存器: ra, sp, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11 (callee-saved registers)
证据	[1] kernel/swtch.S · swtch sd ra, 0(a0); sd sp, 8(a0); sd s0, 16(a0); ... sd s11, 104(a0); ld ra, 0(a1); ... ld s11, 104(a1) [2] kernel/include/context.h · context typedef struct context { uint64 ra; uint64 sp; uint64 s0-s11; } context;	[1] kernel/swtch.S · swtch swtch: sd ra, 0(a0) sd sp, 8(a0) sd s0, 16(a0) sd s1, 24(a0) sd s2, 32(a0) ... sd s11, 104(a0)

Q04_005 · short_answer

调度算法 (Scheduling Algorithm) 属于哪类？请按格式作答：- 算法名称: **(必须是以下之一: FCFS / Round-Robin (RR) / Stride/Proportional-Share / MLFQ / CFS / Priority / 其他)** - 代码证据 (关键字段/函数) : __ - RR: timeslice/slice 字段位置= - Stride: stride 字段与比较逻辑位置= - MLFQ: 多级队列 VecDeque/数组层级证据= - Priority: priority 字段参与 pick_next 排序证据=__

项目	方法A	方法B
答案	算法名称: Round-Robin (RR) 代码证据: - scheduler() 按 proc 数组顺序扫描 RUNNABLE 进程 (kernel/proc.c:673-743) - 时钟中断触发 yield() 实现时间片轮转 (kernel/trap.c:188-191) - 无 priority/stride/timeslice 字段, 采用 FIFO 顺序扫描 + 时钟抢占	算法名称: Round-Robin (RR) 代码证据: scheduler 函数遍历 proc 数组, 选择第一个 RUNNABLE 进程, 无优先级排序, 时间片由时钟中断强制 yield
证据	[1] kernel/proc.c · scheduler for (p = proc; p < &proc[NPROC]; p++) { if (p->state == RUNNABLE) { ... } } [2] kernel/trap.c · kerneltrap if (which_dev == 2 && myproc() != 0 && myproc()->state == RUNNING) { yield(); }	[1] kernel/proc.c · scheduler void scheduler(void) { struct proc p; struct cpu c = mycpu(); ... for(p = proc; p < &proc[NPROC]; p++) { acquire(&p->lock); if(p->state == RUNNABLE) { ... swtch(&c->context, &p->context); ...

Q04_006 · short_answer

调度器 (Scheduler)核心入口/关键函数有哪些？ (schedule/pick_next 等; 给 1-3 个入口与证据)

项目	方法A	方法B
答案	1. scheduler() - 主调度循环 (kernel/proc.c:669-760) 2. sched() - 触发上下文切换 (kernel/proc.c:762-781) 3. yield() - 主动让出 CPU (kernel/proc.c:784-792)	核心入口: scheduler() (kernel/proc.c) 关键函数: yield() (kernel/proc.c), sched() (kernel/proc.c)
证据	[1] kernel/proc.c · scheduler void scheduler(void) { for (;;) { for (p = proc; p < &proc[NPROC]; p++) { if (p->state == RUNNABLE) { ... } } } } [2] kernel/proc.c · sched void sched(void) { swtch(&p->context, &mycpu()->context); } [3] kernel/proc.c · yield void yield(void) { p->state = RUNNABLE; p->main_thread->state = t_RUNNABLE; sched(); }	[1] kernel/proc.c · scheduler void scheduler(void) { ... } [2] kernel/proc.c · yield void yield(void) { ... sched(); ... }

Q04_008 · short_answer

fork/clone 是否复制地址空间与文件表？ (必须给复制路径证据; 若 stub 需说明形态)

项目	方法A	方法B
答案	fork 复制地址空间与文件表: - 地址空间: uvmcopy() 复制页表 (kernel/proc.c:454) - 文件表: filedup() 复制文件描述符 (kernel/proc.c:505-507) - 当前目录: edup() 复制 cwd (kernel/proc.c:508) clone (线程) 共享地址空间: - thread_clone() 共享 p->pagetable (kernel/proc.c:1073-1120) - 不复制文件表, 共享父进程资源	是, fork/clone 调用 uvmcopy 复制地址空间, 并循环复制文件描述符 (ofile)

项目	方法A	方法B
证据	<pre>[1] kernel/proc.c · fork if (uvmcopy(p->pagetable, np->pagetable, np->kpagetable, p->sz) < 0) { ... } for (i = 0; i < NOFILE; i++) if (p->ofile[i]) np->ofile[i] = filedup(p->ofile[i]); [2] kernel/proc.c · thread_clone t->p = p; t->trapframe->sp = stackVa; // 共享进程地址空间</pre>	<pre>[1] kernel/proc.c · fork if (uvmcopy(p->pagetable, np->pagetable, np->kpagetable, p->sz) < 0) ... for (i = 0; i < NOFILE; i++) if (p->ofile[i]) np->ofile[i] = filedup(p->ofile[i]);</pre>

Q04_012 · short_answer

PID 分配器实现是什么？（自增/bitmap/空闲栈复用/只分配不回收；必须给证据）

项目	方法A	方法B
答案	单调自增分配器: nextpid 全局变量 (kernel/proc.c:27), 通过 allocpid() 加锁递增 (kernel/proc.c:142-150) 无回收机制, PID 只分配不回收	自增计数器 (nextpid), 受 pid_lock 保护
证据	<pre>[1] kernel/proc.c · allocpid int allocpid() { acquire(&pid_lock); pid = nextpid; nextpid = nextpid + 1; release(&pid_lock); return pid; } [2] kernel/proc.c · nextpid int nextpid = 1;</pre>	<pre>[1] kernel/proc.c · allocpid int allocpid() { int pid; acquire(&pid_lock); pid = nextpid; nextpid = nextpid + 1; release(&pid_lock); return pid; }</pre>

Q04_013 · short_answer

父子进程树如何存储？（children Vec/链表/parent+sibling 指针；必须给结构体字段证据）

项目	方法A	方法B
答案	单向 parent 指针: struct proc::parent (kernel/include/proc.h:62) 遍历方式: 全局 proc 数组线性扫描 (kernel/proc.c:521-540 reparent(), kernel/proc.c:623-648 wait()) 无 children 链表或 sibling 指针, 查找子进程需 O(NPROC) 扫描	parent 指针 (struct proc *parent), reparent 函数遍历全局 proc 数组查找子进程
证据	<pre>[1] kernel/include/proc.h · proc struct proc { struct proc *parent; ... } [2] kernel/proc.c · wait for (np = proc; np < &proc[NPROC]; np++) { if (np->parent == p) { ... } }</pre>	<pre>[1] kernel/include/proc.h · proc struct proc parent; [2] kernel/proc.c · reparent void reparent(struct proc p) { struct proc *pp; for (pp = proc; pp < &proc[NPROC]; pp++) { if (pp->parent == p) { ... } }</pre>

Q04_015 · short_answer

与 09 多核的交叉一致性：是否存在每核队列/任务迁移/IPI resched？（需与第 9 章互指证据或写不适用）

项目	方法A	方法B
答案	每核队列：未发现独立运行队列，全局 proc 数组共享 (kernel/proc.c:23) 任务迁移：未发现实现 IPI resched：未发现 sbi_send_ipi 调用点 (仅定义于 kernel/include/sbi.h:82-84) 结论：单全局队列，无负载均衡机制	不存在每核队列（使用全局 proc 数组），无显式任务迁移，IPI resched 未实现（scheduler 轮询全局）
证据	[1] kernel/proc.c · proc struct proc proc[NPROC]; // 全局进程数组 [2] kernel/include/sbi.h · sbi_send_ipi static inline void sbi_send_ipi(unsigned long hart_mask, unsigned long hart_mask_base) { SBI_CALL_2(...); } // 仅定义，未见调用	[1] kernel/proc.c · scheduler for(p = proc; p < &proc[NPROC]; p++) { ... }

Q04_016 · short_answer

exit() 资源回收路径：调用链是什么？是否真正回收地址空间/文件表/通知父进程？（必须给调用链证据；桩则说明）

项目	方法A	方法B
答案	调用链: exit() → reparent() → wakeup1(initproc) → wakeup1(original_parent) → sched() 回收内容: - 文件表: fileclose() 关闭所有打开文件 (kernel/proc.c:553-559) - 当前目录: eput(p->cwd) (kernel/proc.c:561) - 地址空间: freeproc() 释放页表 (kernel/proc.c:287-334) - 通知父进程: wakeup1(original_parent) 唤醒 (kernel/proc.c:592-594) 状态流转: p->state = ZOMBIE (kernel/proc.c:600)	调用链: exit -> fileclose (文件表), uvmfree (地址空间), wakeup1 (通知父进程), sched
证据	[1] kernel/proc.c · exit void exit(int status) { for (fd = 0; fd < NOFILE; fd++) fileclose(fd); eput(p->cwd); wakeup1(original_parent); p->state = ZOMBIE; sched(); } [2] kernel/proc.c · freeproc static void freeproc(struct proc *p) { kfree(p->trapframe); kvmfree(p->kpagetable); proc_freepagetable(p->pagetable, p->sz); ... }	[1] kernel/proc.c · exit void exit(int status) { ... Close all open files ... fileclose(fd) ... wakeup1(original_parent); ... sched(); ... } [2] kernel/proc.c · freeproc if (p->pagetable) { free_vma_list(p); proc_freepagetable(p->pagetable, p->sz); }

Q04_017 · tri_state_impl

是否实现进程组/会话（Process Group / Session, pgid/session/set_sid/setpgid）？（必须三态；有则区分真实检查链 vs 仅占位字段）

项目	方法A	方法B
答案	stub	implemented

项目	方法A	方法B
证据	[1] kernel/include/proc.h · proc struct proc { int pgid; ... } // 仅有 pgid 字段 [2] kernel/sysproc.c · sys_setpgid uint64 sys_setpgid(void) { myproc()->pgid = pgid; return 0; } // 仅设置字段，无会话管理 [3] kernel/sysproc.c · sys_getpgid uint64 sys_getpgid(void) { return myproc()->pgid; }	[1] kernel/include/proc.h · proc int pgid; [2] kernel/sysproc.c · sys_setpgid uint64 sys_setpgid(void) { ... myproc()->pgid = pgid; ... }

Q04_018 · tri_state_impl

是否实现 POSIX 资源限制 (rlimit/RLIMIT/getrlimit/setrlimit) ? (必须三态; 若 implemented 需说明支持的资源类型数量及软/硬限制机制)

项目	方法A	方法B
答案	stub	implemented
证据	[1] kernel/include/proc.h · rlimit typedef struct rlimit { uint64 rlim_cur; uint64 rlim_max; } rlimit; [2] kernel/sysproc.c · sys_prlimit64 uint64 sys_prlimit64() { if (opt == 7 && r.rlim_cur == 42) { myproc()->filelimit = 42; } return 0; } // 仅支持单一资源类型 (opt==7)	[1] kernel/include/proc.h · rlimit typedef struct rlimit { uint64 rlim_cur; uint64 rlim_max; } rlimit; [2] kernel/sysproc.c · sys_prlimit64 uint64 sys_prlimit64() { ... }

Q04_023 · multi_choice

调度器是否追踪/优化以下哪些性能指标 (Scheduling Criteria, Stallings Ch9)? (多选; 未发现则留空并在 notes 写 not_found)

项目	方法A	方法B
答案	["未发现调度性能统计"]	[]
证据	[1] kernel/proc.c · scheduling_metrics_scan 搜索 ktime/utime 字段 (kernel/include/proc.h:93-94), 但未见用于调度决策或统计报告	[1] kernel/proc.c · scheduler 在附件中未找到 CPU 利用率、吞吐量等性能统计代码。

Q04_030 · tri_state_impl

是否实现 IPI (核间中断) 发送与处理? (必须三态)

项目	方法A	方法B
答案	stub	implemented
证据	[1] kernel/include/sbi.h · sbi_send_ipi static inline void sbi_send_ipi(unsigned long hart_mask, unsigned long hart_mask_base) { SBI_CALL_2(SBI_IPI_EXTION, SBI_SEND_IPI, ...); } // 仅定义, 未见调用	[1] kernel/include/sbi.h · sbi_send_ipi static inline void sbi_send_ipi(unsigned long hart_mask, unsigned long hart_mask_base) { ... }

Q04_031 · short_answer

若存在 IPI：发送与处理路径分别在哪些函数/文件？（给关键入口与证据）

项目	方法A	方法B
答案	发送路径: kernel/include/sbi.h:82-84 sbi_send_ipi() (仅定义, 未见调用) 处理路径: 未发现 IPI 处理函数 注: IPI 功能仅定义接口, 未在代码中实际使用	发送: sbi_send_ipi (kernel/include/sbi.h) 处理: devintr (kernel/trap.c)
证据	[1] kernel/include/sbi.h · sbi_send_ipi static inline void sbi_send_ipi(unsigned long hart_mask, unsigned long hart_mask_base) { SBI_CALL_2(SBI_IPI_EXTION, SBI_SEND_IPI, hart_mask, hart_mask_base); }	[1] kernel/include/sbi.h · sbi_send_ipi SBI_CALL_2(SBI_IPI_EXTION, SBI_SEND_IPI, hart_mask, hart_mask_base); [2] kernel/trap.c · devintr int devintr(void) { ... }

Q04_033 · short_answer

per-CPU 的实现方式是什么？（例如 TLS/tp 寄存器/gsbase/数组索引 hartid；需证据）

项目	方法A	方法B
答案	数组索引 + tp 寄存器： - cpus[NCPU] 数组存储每核状态 (kernel/proc.c:21) - mycpu() 通过 tp 寄存器获取 hartid 索引 (kernel/proc.c:134-138) - tp 寄存器在 main() 初始化时设置 (kernel/main.c:28-30)	通过 tp 寄存器存储 hartid, 索引 cpus 数组 (mycpu 函数)
证据	[1] kernel/main.c · inithartid static inline void inithartid(unsigned long hartid) { asm volatile("mv tp, %0" : : "r"(hartid & 0x1)); } [2] kernel/proc.c · mycpu struct cpu* mycpu() { return &cpus[r_tp()]; } // 通过 tp 寄存器索引	[1] kernel/include/riscv.h · r_tp static inline uint64 r_tp() { ... asm volatile("mv %0, tp" ...); } [2] kernel/proc.c · mycpu struct cpu mycpu(void) { int id = cpuid(); struct cpu c = &cpus[id]; ... }

Q04_036 · short_answer

与 03/04/05/08 章的交叉一致性 (Cross-Chapter Consistency), 按以下四项分别作答（每项须给证据路径或写「单核不适用」）：
- 03 TLB: 多核页表修改后 TLB 刷新策略= - 04 调度: 每核运行队列/负载均衡/IPI resched= - 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序= - 08 锁: SpinLock 关中断行为在多核下是否安全=

项目	方法A	方法B
答案	03 TLB: 多核页表修改后 TLB 刷新策略 = 仅本地 sfence_vma(), 未见跨核刷新机制 04 调度: 每核运行队列/负载均衡/IPI resched = 全局 proc 数组, 无每核队列, 无负载均衡, IPI 未使用 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序 = trapinithart() 在从核上线前调用 (kernel/main.c:85) 08 锁: SpinLock 关中断行为在多核下是否安全 = acquire() 调用 push_off() 关中断 (kernel/spinlock.c:20), 多核下安全	03 TLB: 多核页表修改后 TLB 刷新策略=本地 sfence_vma (切换时) 04 调度: 每核运行队列/负载均衡/IPI resched=全局队列, 无负载均衡 05 Trap: per-CPU trap 栈/时钟中断初始化与 AP 上线顺序=trapinithart 每核调用 08 锁: SpinLock 关中断行为在多核下是否安全=安全 (push_off 关中断)

项目	方法A	方法B
证据	[1] kernel/main.c · main else { while (started == 0); kvminithart(); trapinithart(); plicinithart(); ... } [2] kernel/spinlock.c · acquire void acquire(struct spinlock *lk) { push_off(); ... }	[1] kernel/proc.c · scheduler w_satp(...); sfence_vma(); [2] kernel/spinlock.c · acquire push_off(); // disable interrupts to avoid deadlock.

Q04_038 · short_answer

NCPU/MAXCPU（或等价宏）与链接脚本中的每 hart 栈/入口布局是否对应？（搜索_max_hart_id 等；给宏定义与链接脚本对应证据，或写未发现）

项目	方法A	方法B
答案	NCPU=2 (kernel/include/param.h:5) 链接脚本 linker/qemu.ld 未显式定义每 hart 栈布局 从核栈通过全局 cpus[NCPU].context 隐式分配 (kernel/proc.c:21) 未见_max_hart_id 或显式 hart 栈定义	NCPU=2 (kernel/include/param.h)，链接脚本未在提供的文本中显示，但 Makefile 引用了 linker/visionfive.ld
证据	[1] kernel/include/param.h · NCPU #define NCPU 2 // maximum number of CPUs [2] linker/qemu.ld · qemu.ld 链接脚本未定义 hart 栈，仅定义 kernel_start/text_start/data_start/bss_start 等 段	[1] kernel/include/param.h · NCPU #define NCPU 2 [2] Makefile · Makefile linker = ./linker/visionfive.ld

05_fs_drivers — 文件系统与设备 I/O

Q05_001 · short_answer

VFS 抽象层 (Virtual File System, VFS)接口是什么形态？（Rust trait / C op 表；必须给接口定义证据）

项目	方法A	方法B
答案	C 语言设备开关表 (devsw) + 文件结构体 (struct file) 形态。VFS 通过 struct devsw 设备开关表（kernel/include/file.h:41-46）定义设备操作接口（read/write 函数指针），通过 struct file（kernel/include/file.h:16-32）统一表示文件/管道/设备/Socket，type 字段区分 FD_ENTRY/FD_PIPE/FD_DEVICE/FD_SOCK。无传统 Unix inode_operations/file_operations 结构，dirent 结构体（kernel/include/fat32.h:38-65）兼具 inode 功能（含锁、引用计数、簇信息）。	C 操作表 (C operation table)。VFS 通过 struct file 和 struct devsw 等结构体定义文件操作接口，使用函数指针表形式。

项目	方法A	方法B
证据	[1] kernel/include/file.h · devsw struct devsw { char name[DEV_NAME_MAX+1]; struct spinlock lk; int (read)(int, uint64, int); int (write)(int, uint64, int); }; (行 41-46) [2] kernel/include/file.h · file struct file { enum { FD_NONE, FD_PIPE, FD_ENTRY, FD_DEVICE, FD_SOCKET, FD_NULL} type; ... struct dirent *ep; ... }; (行 16-32) [3] kernel/include/fat32.h · dirent struct dirent { char filename[256]; uint8 attribute; uint32 first_clus; uint32 file_size; struct sleeplock lock; int ref; ... }; (行 38-65)	[1] kernel/include/file.h · file struct file { enum { FD_NONE, FD_PIPE, FD_ENTRY, FD_DEVICE, FD_SOCKET, FD_NULL} type; int ref; char readable; char writable; struct pipe pipe; struct dirent ep; uint off; short major; struct socket sock; ...}; [2] kernel/include/file.h · devsw struct devsw { char name[DEV_NAME_MAX+1]; struct spinlock lk; int (read)(int, uint64, int); int (write)(int, uint64, int); }; [3] kernel/file.c · fileread int fileread(struct file f, uint64 addr, int n) { ... switch (f->type) { case FD_PIPE: r = piperead(f->pipe, 1, addr, n); break; case FD_DEVICE: r = devsw[f->major].read(1, addr, n); break; case FD_ENTRY: r = eread(f->ep, 1, addr, f->off, n); break; ... }}

Q05_003 · short_answer

若支持 FAT32/Ext4：它是自研还是第三方库/crate？（必须引用 Cargo.toml/Cargo.lock 或 Makefile 引入证据）

项目	方法A	方法B
答案	自研实现。项目为 C/Makefile 风格（非 Rust），FAT32 文件系统代码位于 kernel/fat32.c（1184 行）和 kernel/include/fat32.h，无第三方库引入证据。Makefile（kernel/net.mk）仅引入 lwip 网络栈，文件系统为内核原生实现。	自研实现。FAT32 文件系统在 kernel/fat32.c 中直接实现，没有使用第三方库。
证据	[1] kernel/fat32.c · fat32.c 1184 行 FAT32 实现，包含 eget/ename/dirlookup 等核心函数 [2] Makefile · Makefile OBSJS += \$K/fat32.o（行 40），直接编译内核 FAT32 模块	[1] Makefile · OBJJS OBJJS += ... \$K/fat32.o \$K/fs.o ... [2] kernel/fat32.c · fat32.c // FAT32 filesystem implementation - contains struct dirent, fat32_init(), dirlookup(), eread(), ewrite() etc. [3] kernel/include/fat32.h · fat32.h #ifndef __FAT32_H ... struct dirent { char filename[FAT32_MAX_FILENAME + 1]; uint8 attribute; uint32 first_clus; uint32 file_size; ... };

Q05_004 · short_answer

文件打开路径：文件打开入口（sys_open 或等价）→ VFS 层 → 具体 FS open。列出 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	文件打开路径: 1. sys_open (kernel/sysfile.c:455) → 2. open (kernel/sysfile.c:410) → 3. ename/create (FAT32 目录查找/创建) → 4. filealloc (kernel/file.c:51) 分配 file 结构 → 5. fdalloc (kernel/sysfile.c:69) 分配 fd → 6. 返回 fd。关键调用: sys_open 调用 argstr 获取路径参数, 调用 open 函数; open 调用 ename 查找 dirent 或 create 创建文件, 调用 filealloc 和 fdalloc 分配资源。	文件打开路径关键节点: 1) sys_open (syscall 入口) 2) open (VFS 层) 3) ename (路径解析) 4) filealloc (文件结构分配) 5) fdalloc (文件描述符分配)
证据	[1] kernel/sysfile.c · sys_open <pre>uint64 sys_open(void) { argstr(0, path, ...); return open(path, omode); } (行 455-460)</pre> [2] kernel/sysfile.c · open <pre>uint64 open(char path, int omode) { ep = ename(path); f = filealloc(); fd = fdalloc(f); f->type = FD_ENTRY; f->ep = ep; return fd; } (行 410-453)</pre> [3] kernel/file.c · filealloc <pre>struct file filealloc(void) { acquire(&ftable.lock); for(f = ftable.file; ...) if(f->ref == 0) { f->ref = 1; return f; } } (行 51-63)</pre>	[1] kernel/sysfile.c · sys_open <pre>uint64 sys_open(void) { char path[FAT32_MAX_PATH]; int omode; if (argstr(0, path, FAT32_MAX_PATH) < 0 argint(1, &omode) < 0) return -1; return open(path, omode); }</pre> [2] kernel/sysfile.c · open <pre>uint64 open(char path, int omode) { ... if (omode & O_CREATE) { ep = create(path, T_FILE, omode); } else { if ((ep = ename(path)) == NULL) { return -1; } ... } if ((f = filealloc()) == NULL (fd = fdalloc(f)) < 0) { ... } f->type = FD_ENTRY; f->ep = ep; ... }</pre> [3] kernel/fat32.c · ename <pre>struct dirent ename(char path) { char name[FAT32_MAX_FILENAME + 1]; return lookup_path(path, 0, name); }</pre> [4] kernel/file.c · filealloc <pre>struct file filealloc(void) { struct file *f; acquire(&ftable.lock); for (f = ftable.file; f < ftable.file + NFILE; f++) { if (f->ref == 0) { f->ref = 1; release(&ftable.lock); return f; } } release(&ftable.lock); return NULL; }</pre>

Q05_005 · short_answer

文件描述符表 (File Descriptor Table, FD Table) 的实现形态是什么？（固定数组/Vec/BTreeMap 等；必须给结构体定义证据）

项目	方法A	方法B
答案	Per-Process 固定数组。每个进程 struct proc 包含 struct file *ofile[NFILE] (kernel/include/proc.h:81), NFILE=128 (kernel/include/param.h:6)。fdalloc 函数 (kernel/sysfile.c:69-80) 线性扫描 p->ofile 数组寻找空闲槽位。	固定数组实现。每个进程的 ofile 数组存储文件描述符, 大小为 NFILE (128)。

项目	方法A	方法B
证据	<pre>[1] kernel/include/proc.h · proc struct proc { ... struct file ofile[NOFILE]; ... }; (行 81) [2] kernel/include/param.h · NOFILE #define NOFILE 128 // open files per process (行 6) [3] kernel/sysfile.c · fdalloc static int fdalloc(struct file f) { for(fd = 0; fd < NOFILEMAX(p); fd++) if(p->ofile[fd] == 0) { p->ofile[fd] = f; return fd; } } (行 69-80)</pre>	<pre>[1] kernel/include/proc.h · proc struct proc { ... struct file ofile[NOFILE]; // Open files ... }; [2] kernel/include/param.h · NOFILE #define NOFILE 128 // open files per process [3] kernel/sysfile.c · fdalloc static int fdalloc(struct file f) { int fd; struct proc *p = myproc(); for (fd = 0; fd < NOFILEMAX(p); fd++) { if (p->ofile[fd] == 0) { p->ofile[fd] = f; return fd; } } return -1; }</pre>

Q05_007 · short_answer

若存在缓存：驱逐策略是什么（LRU/Clock/FIFO/无驱逐）？必须指出判断依据（字段/算法分支）证据。

项目	方法A	方法B
答案	LRU（最近最少使用）驱逐策略。bcache 结构（kernel/bio.c:28-34）维护双向链表，head.next 为最近使用，head.prev 为最久未使用。bget 函数（kernel/bio.c:62-95）从 head.prev 开始查找 refcnt == 0 的缓冲进行回收。brelse 函数（kernel/bio.c:115-135）将释放的缓冲移到 head.next（最近使用端）。	LRU（最近最少使用）策略。bcache 使用双向链表组织 buffer，最近使用的 buffer 排在队头，驱逐时从队尾查找 refcnt==0 的空闲 buffer。
证据	<pre>[1] kernel/bio.c · bcache struct { ... struct buf head; } bcache; // head.next 最近使用，head.prev 最久未使用 (行 28-34) [2] kernel/bio.c · bget for(b = bcache.head.prev; b != &bcache.head; b = b->prev) if(b->refcnt == 0) { ... return b; } (行 77-89), 从 LRU 端回收 [3] kernel/bio.c · brelse b->next = bcache.head.next; b->prev = &bcache.head; bcache.head.next->prev = b; bcache.head.next = b; (行 125-130), 移到 MRU 端</pre>	<pre>[1] kernel/bio.c · bget // Recycle the least recently used (LRU) unused buffer. for (b = bcache.head.prev; b != &bcache.head; b = b->prev) { if (b->refcnt == 0) { ... return b; } } [2] kernel/bio.c · brelse void brelse(struct buf *b) { ... b->refcnt--; if (b->refcnt == 0) { b->next->prev = b->prev; b->prev->next = b->next; b->next = bcache.head.next; b->prev = &bcache.head; bcache.head.next->prev = b; bcache.head.next = b; } ... } [3] doc/buffer_cache.md · buffer_cache 这一队列遵循类似于最近最少使用（LRU）的替换方法，刚刚使用的 buffer cache 会排到队头。</pre>

Q05_008 · tri_state_impl

是否实现页缓存 (Page Cache)或与 mmap/文件映射共享缓存页？（必须三态）

项目	方法A	方法B
答案	implemented	not_found

项目	方法A	方法B
证据	[1] kernel/mmap.c · mmap mmap 通过 vma 管理文件映射，调用 fileread 读取文件内容到内存页（行 12-60） [2] kernel/include/vma.h · vma struct vma { enum segtype type; int perm; uint64 addr; uint64 sz; int fd; uint64 f_off; ... };（行 14-25）	[1] kernel/mmap.c · page_cache 在附件源码中搜索 page cache 相关实现，未找到独立的页缓存实现。mmap 直接读取文件内容到分配的页面，没有共享缓存机制。 [2] kernel/mmap.c · mmap uint64 mmap(...) { ... for (int i = 0; i < page_n; i++) { ... fileread(f, va, PGSIZE); ... } ... }

Q05_012 · tri_state_impl

是否支持符号链接 (symlink) 的解析/跟随？（必须三态）

项目	方法A	方法B
答案	stub	not_found
证据	[1] kernel/sysfile.c · sys_readlinkat uint64 sys_readlinkat(void) { ... if(strncmp(path, "/proc/self/exe", 14) == 0) { either_copyout(1, addr2, "/", 1); ... } }（行 1347-1380），仅硬编码处理 /proc/self/exe，无通用 symlink 实现 [2] kernel/include/fat32.h · ATTR_VOLUME_ID 无 ATTR_SYMLINK 定义，FAT32 本身不支持 symlink 属性	[1] kernel/fat32.c · symlink 在附件源码中搜索 symlink 相关实现，未找到符号链接的解析或跟随实现。FAT32 文件系统本身不支持符号链接。 [2] kernel/include/fat32.h · symlink 在 fat32.h 头文件中未找到符号链接相关的结构体或函数定义。

Q05_023 · short_answer

驱动框架接口是什么？（Rust Driver trait / C driver ops / 注册表；必须引用接口定义证据）

项目	方法A	方法B
答案	C 语言设备开关表 (devsw) + 条件编译平台适配。struct devsw（kernel/include/file.h:41-46）定义设备操作接口（read/write 函数指针）。平台适配通过 #ifdef QEMU / #ifdef visionfive 条件编译选择不同驱动（kernel/disk.c:9-45 选择 virtio 或 SDIO）。	C 驱动操作表 (C driver ops)。通过 devsw 结构体定义设备驱动接口，包含 read/write 函数指针。
证据	[1] kernel/include/file.h · devsw struct devsw { char name[DEV_NAME_MAX+1]; int (read)(int, uint64, int); int (write)(int, uint64, int); };（行 41-46） [2] kernel/disk.c · disk.c #ifdef QEMU virtio_disk_init(); #else ... #endif（行 9-45），条件编译选择驱动	[1] kernel/include/file.h · devsw struct devsw { char name[DEV_NAME_MAX+1]; struct spinlock lk; int (read)(int, uint64, int); int (write)(int, uint64, int); }; [2] kernel/include/file.h · devsw extern struct devsw devsw[]; [3] kernel/console.c · consoleinit void consoleinit(void) { ... devsw[CONSOLE].read = consoleread; devsw[CONSOLE].write = consolewrite; }

Q05_024 · short_answer

驱动注册与初始化顺序是什么？（init_drivers/probe/driver_manager 等；列出 3-6 个关键节点并给证据）

项目	方法A	方法B
答案	初始化顺序 (kernel/main.c:55-70) : 1. kinit() 物理页分配器 → 2. kvminit() 内核页表 → 3. plicinit() 中断控制器 → 4. disk_init() 磁盘驱动 (kernel/disk.c:15 调用 virtio_disk_init 或 sd_init) → 5. binit() 块缓存 → 6. fileinit() 文件表 → 7. consoleinit() 控制台。驱动通过 Makefile 条件编译 (Makefile:60-75) 选择 virtio_disk.o 或 sd_final.o 链接。	驱动初始化顺序: 1) main() 2) consoleinit() 3) disk_init() 4) 各设备驱动初始化函数
证据	[1] kernel/main.c · main <pre>plicinit(); disk_init(); binit(); fileinit(); ... (行 60-68), 初始化顺序</pre> [2] kernel/disk.c · disk_init <pre>void disk_init(void) { #ifdef QEMU virtio_disk_init(); #else ... #endif } (行 9-18)</pre> [3] Makefile · Makefile <pre>ifeq (\$(platform), qemu) OBJS += \$K/virtio_disk.o; else OBJS += \$K/sd_final.o; endif (行 60-75)</pre>	[1] kernel/main.c · main <pre>void main(...) { ... consoleinit(); ... disk_init(); ... }</pre> [2] kernel/console.c · consoleinit <pre>void consoleinit(void) { initlock(&cons.lock, "cons"); #ifdef QEMU uartinit(); #endif #ifdef visionfive uart8250_init(UART, 24000000, 115200, 2, 4, 0); #endif ... devsw[CONSOLE].read = consoleread; devsw[CONSOLE].write = consolewrite; }</pre> [3] kernel/disk.c · disk_init <pre>void disk_init(void) { #ifdef QEMU virtio_disk_init(); #else ramdisk_init(); #endif }</pre>

Q05_029 · short_answer

MMIO 地址来源是什么？（DTB 提供 / 常量硬编码 / 物理→虚拟转换；必须给证据）

项目	方法A	方法B
答案	常量硬编码 + 物理→虚拟转换。 kernel/include/memlayout.h:43-44 定义 #define UART 0x10000000L 和 #define UART_V (UART + VIRT_OFFSET)，其中 VIRT_OFFSET=0x3F0000000L（行 40）。驱动代码使用 UART_V 访问虚拟地址。	常量硬编码。MMIO 地址在 memlayout.h 中定义为常量宏，通过虚拟地址偏移转换为虚拟地址。
证据	[1] kernel/include/memlayout.h · UART <pre>#define UART 0x10000000L (行 43), 物理地址硬编码</pre> [2] kernel/include/memlayout.h · UART_V <pre>#define UART_V (UART + VIRT_OFFSET) (行 44), 虚拟地址 = 物理地址 + 偏移</pre>	[1] kernel/include/memlayout.h · UART <pre>#define VIRT_OFFSET 0x3F0000000L #define UART 0x10000000L #define UART_V (UART + VIRT_OFFSET) #define CLINT 0x02000000L #define CLINT_V (CLINT + VIRT_OFFSET) #define PLIC 0x0c000000L #define PLIC_V (PLIC + VIRT_OFFSET)</pre> [2] kernel/vm.c · kvminit <pre>void kvminit() { ... #ifdef visionfive kvmmap(UART_V, UART, 0x10000, PTE_R PTE_W); #endif #ifdef QEMU kvmmap(VIRTIO0_V, VIRTIO0, PGSIZE, PTE_R PTE_W); #endif ... }</pre>

Q05_030 · short_answer

多平台适配是如何通过构建/条件编译选择驱动的？（features/Kconfig/Makefile 规则；必须给证据）

项目	方法A	方法B
答案	Makefile 条件编译 + #ifdef 平台宏。Makefile:1-2 设置 platform := visionfive 或 qemu，Makefile:60-75 根据平台选择 virtio_disk.o 或 sd_final.o。源码使用 #ifdef QEMU / #ifdef visionfive (kernel/disk.c:9-45、 kernel/plic.c:26-45) 区分平台逻辑。	通过 Makefile 中的 platform 变量和条件编译实现多平台适配。支持 visionfive 和 qemu 平台。
证据	[1] Makefile · Makefile platform := visionfive (行 1); ifeq (\$(platform), qemu) OBJS += \$K/virtio_disk.o; else OBJS += \$K/sd_final.o; endif (行 60-75) [2] kernel/disk.c · disk.c #ifdef QEMU virtio_disk_init(); #else ... #endif (行 9-45)	[1] Makefile · platform platform := visionfive #platform := qemu ... ifeq (\$(platform), visionfive) OBJS += \$K/entry_visionfive.o else OBJS += \$K/entry_qemu.o endif [2] Makefile · CFLAGS ifeq (\$(platform), qemu) CFLAGS += -D QEMU else ifeq (\$(platform), visionfive) CFLAGS += -D visionfive OBJS += \$K/sddata.o \$K/ramdisk.o endif [3] kernel/disk.c · ifdef void disk_init(void) { #ifdef QEMU virtio_disk_init(); #else ramdisk_init(); #endif }

Q05_033 · single_choice

块设备（磁盘/eMMC/NVMe）I/O 请求调度算法 (Scheduling Algorithm) (Disk Scheduling Algorithm) 更接近哪种？ (Stallings Ch11；若无显式调度则选「FCFS 顺序提交」)

项目	方法A	方法B
答案	E. 基于 virtio 环 (queue 顺序提交，无显式磁盘调度)	A. FCFS (先来先服务 First-Come First-Served)
证据	[1] kernel/virtio_disk.c · virtio_disk_rw 分配 virtio 描述符环 (行 163-210)，按提交顺序放入环中，无 SCAN/SSTF 等调度算法 [2] kernel/bio.c · bread b = bget(dev, sectorno); if(!b->valid) disk_read(b); (行 97-105)，按请求顺序提交，无重排序	[1] kernel/virtio_disk.c · schedule 在附件源码中搜索磁盘调度算法相关实现，未找到 SSTF/SCAN/C-SCAN 等高级调度算法实现。请求按提交顺序处理。 [2] kernel/virtio_disk.c · virtio_disk_rw void virtio_disk_rw(struct buf b, int write) { ... disk.avail[2 + (disk.avail[1] % NUM)] = idx[0]; __sync_synchronize(); disk.avail[1] = disk.avail[1] + 1; R(VIRTIO_MMIO_QUEUE_NOTIFY) = 0; ... }

06_sync_ipc — 同步互斥与进程间通信

Q06_001 · short_answer

该内核提供了哪些同步原语？ (SpinLock/Mutex/RwLock/Semaphore/Condvar/WaitQueue 等；列出类型定义证据)

项目	方法A	方法B
答案	已实现以下同步原语: 1. SpinLock (自旋锁) : <code>kernel/include/spinlock.h:7-12</code> 定义 <code>struct spinlock { uint locked; char *name; struct cpu *cpu; }</code>, 实现于 <code>kernel/spinlock.c:20</code> 的 <code>acquire()</code> 和 <code>kernel/spinlock.c:50</code> 的 <code>release()</code>。 2. SleepLock (睡眠锁) : <code>kernel/include/sleeplock.h</code> 定义 <code>struct sleeplock { struct spinlock lk; int locked; int pid; char *name; }</code>, 实现于 <code>kernel/sleeplock.c:18</code> 的 <code>acquiresleep()</code> 和 <code>kernel/sleeplock.c:28</code> 的 <code>releasesleep()</code>。 3. Semaphore (信号量) : <code>kernel/include/sem.h:7-11</code> 定义 <code>struct semaphore { int value; int valid; struct spinlock lock; }</code>, 实现于 <code>kernel/sem.c:16</code> 的 <code>sem_wait()</code> 和 <code>kernel/sem.c:44</code> 的 <code>sem_post()</code>。 4. Futex WaitQueue (Futex 等待队列) : <code>kernel/futex.c:8-12</code> 定义 <code>struct FutexQueue { uint64 addr; thread *thread; uint8 valid; }</code>, 实现于 <code>kernel/futex.c:16</code> 的 <code>futexWait()</code> 和 <code>kernel/futex.c:38</code> 的 <code>futexWake()</code>。 5. WaitQueue (通过 sleep/wakeup 实现) : <code>kernel/proc.c:818</code> 的 <code>sleep()</code> 和 <code>kernel/proc.c:851</code> 的 <code>wakeup()</code> 提供基于通道 (chan) 的阻塞/唤醒机制。 6. Pipe (管道) : <code>kernel/include/pipe.h:10-17</code> 定义 <code>struct pipe { struct spinlock lock; char data[PIPESIZE]; uint nread; uint nwrite; int readopen; int writeopen; }</code>, 实现于 <code>kernel/pipe.c:13</code> 的 <code>pipealloc()</code>、<code>kernel/pipe.c:63</code> 的 <code>pipewrite()</code> 和 <code>kernel/pipe.c:90</code> 的 <code>piperead()</code>。 未发现: <code>RwLock</code> (读写锁)、<code>Condvar</code> (条件变量)、<code>Monitor</code> (管程)、<code>Barrier</code> (屏障同步)。	<code>SpinLock</code> (<code>kernel/include/spinlock.h</code>), <code>Semaphore</code> (<code>kernel/include/sem.h</code>), <code>SleepLock/Mutex</code> (<code>kernel/include/sleeplock.h</code>). <code>WaitQueue</code> functionality is provided via <code>sleep/wakeup</code> channels in <code>kernel/proc.c</code>.
证据	[1] <code>kernel/include/spinlock.h · spinlock</code> <code>struct spinlock { uint locked; char name; struct cpu cpu; };</code> [2] <code>kernel/include/sem.h · semaphore</code> <code>struct semaphore { int value; int valid; struct spinlock lock; };</code> [3] <code>kernel/futex.c · FutexQueue</code> <code>typedef struct FutexQueue { uint64 addr; thread *thread; uint8 valid; } FutexQueue;</code> [4] <code>kernel/include/pipe.h · pipe</code> <code>struct pipe { struct spinlock lock; char data[PIPESIZE]; uint nread; uint nwrite; int readopen; int writeopen; };</code>	[1] <code>kernel/include/spinlock.h · spinlock</code> <code>struct spinlock { uint locked; char name; struct cpu cpu; };</code> [2] <code>kernel/include/sem.h · semaphore</code> <code>struct semaphore { int value; int valid; struct spinlock lock; };</code> [3] <code>kernel/include/sleeplock.h · sleeplock</code> <code>struct sleeplock { uint locked; struct spinlock lk; char name; int pid; };</code> [4] <code>kernel/proc.c · sleep</code> <code>void sleep(void chan, struct spinlock *lk) { ... }</code>

Q06_004 · fill_in

sleep / wakeup 不变量 (Sleep-Wakeup Invariant) 分析, 按格式填写: - sleep 入口函数: (路径) - 入睡前持有的锁: (无则写 none) - 防丢 wakeup (Lost Wakeup Prevention) 机制: (如: 持队列锁检查条件 / 无防护) - wakeup 函数: (路径) - 唤醒与锁释放顺序: __ (先唤醒后释放 / 先释放后唤醒 / 其他)

项目	方法A	方法B
答案	sleep 入口函数: kernel/proc.c:818 (sleep) 入睡前持有的锁: 调用者传入的 lk (spinlock) , sleep 内部会获取 p->lock 防丢 wakeup (Lost Wakeup Prevention) 机制: 持有 p->lock 后释放 lk, 确保 wakeup 在检查条件时持有 p->lock (见 kernel/proc.c:821-830 注释: "Once we hold p->lock, we can be guaranteed that we won't miss any wakeup") wakeup 函数: kernel/proc.c:851 (wakeup) 唤醒与锁释放顺序: wakeup 遍历 proc 数组, 对每个进程先 acquire(&p->lock), 检查条件后设置 p->state = RUNNABLE, 然后 release(&p->lock) (先唤醒后释放)	sleep 入口函数: kernel/proc.c::sleep 入睡前持有的锁: lk (function argument) 防丢 wakeup (Lost Wakeup Prevention) 机制: sleep 内部获取 p->lock (proc lock) 后再释放 lk 并睡眠; wakeup 也获取 p->lock 后检查条件 wakeup 函数: kernel/proc.c::wakeup 唤醒与锁释放顺序: 先唤醒后释放 (wakeup 中释放 p->lock 前设置状态; 在 sleeplock.c releasesleep 中调用 wakeup 后释放 lk->lk)
证据	[1] kernel/proc.c:818 · sleep <pre>void sleep(void chan, struct spinlock lk) { if (lk != &p->lock) { acquire(&p->lock); release(lk); } p->chan = chan; p->state = SLEEPING; sched(); ... if (lk != &p->lock) { release(&p->lock); acquire(lk); } }</pre> [2] kernel/proc.c:851 · wakeup <pre>void wakeup(void *chan) { for (p = proc; p < &proc[NPROC]; p++) { acquire(&p->lock); if (p->state == SLEEPING && p->chan == chan) { p->state = RUNNABLE; } release(&p->lock); } }</pre>	[1] kernel/proc.c · sleep <pre>void sleep(void chan, struct spinlock lk) { ... if (lk != &p->lock) { acquire(&p->lock); release(lk); } p->chan = chan; p->state = SLEEPING; sched(); ... }</pre> [2] kernel/proc.c · wakeup <pre>void wakeup(void chan) { ... acquire(&p->lock); if (p->state == SLEEPING && p->chan == chan) { p->state = RUNNABLE; } release(&p->lock); ... }</pre> [3] kernel/sleeplock.c · releasesleep <pre>void releasesleep(struct sleeplock lk) { acquire(&lk->lk); lk->locked = 0; wakeup(lk); release(&lk->lk); }</pre>

Q06_008 · tri_state_impl

是否实现消息队列/信号量/共享内存等 SysV IPC (Message Queue / Semaphore / Shared Memory, msg/sem/shm)? (必须三态; 若仅实现其一需说明)

项目	方法A	方法B
答案	stub	not_found
证据	[1] kernel/sem.c · semaphore_impl 信号量已实现: sem_init, sem_wait, sem_post, sem_destroy (kernel/sem.c:8-72)。但 SysV IPC 系统调用 sys_semget/sys_msgget/sys_shmget 未实现 (grep 搜索 0 命中)。 [2] kernel/sysproc.c · sysv_ipc_syscall_scan 在 sysproc.c 和 syscall.c 中检索 sys_msgget/sys_semget/sys_shmget, 0 命中; 仅实现内核态信号量原语, 无用户态 SysV IPC 系统调用。	[1] kernel/include/sem.h · sem.h <pre>struct semaphore { ... }; void sem_init(...); void sem_wait(...); void sem_post(...);</pre> [2] kernel/sem.c · sem.c Implementation of POSIX-like semaphores (sem_init, sem_wait, sem_post), not SysV IPC (semget, semop). [3] kernel/sysproc.c · sys_msgget No SysV IPC system calls (msgget, shmget, semget) found in syscall table.

Q06_011 · short_answer

若实现 signal handler: 用户态 handler 上下文如何构建? 是否存在 sigreturn 恢复原 trap frame? (必须给证据)

项目	方法A	方法B
答案	用户态 handler 上下文构建: - 在 kernel/signal.c:59 的 sighandle() 中, 内核分配 sig_tf = kalloc() 保存当前 trapframe (memcpy(p->sig_tf, p->trapframe, sizeof(struct trapframe))) 。 - 修改 trapframe 的 epc 为用户注册的信号处理函数地址 (p->trapframe->epc = (uint64)p->sigaction[signal].__sigaction_handler.sa_handler) 。 - 设置 ra = SIGTRAMPOLINE (kernel/include/memlayout.h:109 定义 SIGTRAMPOLINE = TRAPFRAME - PGSIZE) , 使 handler 返回后跳转到信号蹦床。 - 调整用户栈指针 sp = sp - PGSIZE 为 handler 提供栈空间。 sigreturn 恢复机制: - kernel/SignalTrampoline.S:4 定义 signalTrampoline 段, 执行 li a7, 139; ecall 触发 139 号系统调用 (SYS_rt_sigreturn) 。 - kernel/sys.sig.c:93 的 sys_rt_sigreturn() 调用 rt_sigreturn() 。 - kernel/signal.c:51 的 rt_sigreturn() 恢复 trapframe (memcpy(p->trapframe, p->sig_tf, sizeof(struct trapframe))) , 释放 sig_tf, 返回原 a0 。 证据路径: kernel/signal.c:59-77 (sighandle 上下文构建) , kernel/signal.c:51-56 (rt_sigreturn 恢复) , kernel/SignalTrampoline.S:1-6 (蹦床代码) , kernel/include/memlayout.h:109 (SIGTRAMPOLINE 地址) 。	Context saved in p->sig_tf (kernel/signal.c::sighandle). Handler entry set in trapframe->epc. Return address set to signalTrampoline. sigreturn (rt_sigreturn) restores trapframe from p->sig_tf.
证据	[1] kernel/signal.c:59 · sighandle p->sig_tf = kalloc(); memcpy(p->sig_tf, p->trapframe, sizeof(struct trapframe)); p->trapframe->epc = (uint64)p->sigaction[signal].__sigaction_handler.sa_handler; p->trapframe->ra = (uint64)SIGTRAMPOLINE; [2] kernel/signal.c:51 · rt_sigreturn uint64 rt_sigreturn(void) { memcpy(p->trapframe, p->sig_tf, sizeof(struct trapframe)); kfree(p->sig_tf); p->sig_tf = 0; return p->trapframe->a0; } [3] kernel/SignalTrampoline.S:1 · signalTrampoline signalTrampoline: li a7, 139; ecall	[1] kernel/signal.c · sighandle p->sig_tf = kalloc(); memcpy(p->sig_tf, p->trapframe, sizeof(struct trapframe)); p->trapframe->epc = (uint64)p->sigaction[signal].__sigaction_handler.sa_handler; p->trapframe->ra = (uint64)SIGTRAMPOLINE; [2] kernel/signal.c · rt_sigreturn uint64 rt_sigreturn(void) { ... memcpy(p->trapframe, p->sig_tf, sizeof(struct trapframe)); kfree(p->sig_tf); ... } [3] kernel/SignalTrampoline.S · signalTrampoline signalTrampoline: li a7, 139; ecall

Q06_013 · single_choice

底层原子操作来源更接近哪种?

项目	方法A	方法B
答案	自定义汇编 (ldxr/stxr、lock xchg 等)	D. 未发现/不确定

项目	方法A	方法B
证据	[1] kernel/spinlock.c:41 · __sync_lock_test_and_set while (__sync_lock_test_and_set(&lk->locked, 1) != 0) ; // On RISC-V, sync_lock_test_and_set turns into an atomic swap: amoswap.w.aq [2] kernel/spinlock.c:76 · __sync_lock_release __sync_lock_release(&lk->locked); // On RISC-V, sync_lock_release turns into an atomic swap: amoswap.w zero, zero, (s1)	[1] kernel/spinlock.c · acquire while (__sync_lock_test_and_set(&lk->locked, 1) != 0) ; [2] kernel/include/riscv.h · atomic Uses GCC built-in __sync functions, not Rust or explicit custom assembly in source.

Q06_014 · short_answer

死锁四必要条件（Coffman Conditions）在该内核中是否均成立？请逐条作答（互斥 Mutual Exclusion / 持有并等待 Hold-and-Wait / 不可剥夺 No Preemption / 循环等待 Circular Wait），并结合 SpinLock/Mutex 的实现给出证据或写「不适用」。

项目	方法A	方法B
答案	1. 互斥 (Mutual Exclusion): 成立。SpinLock 通过 __sync_lock_test_and_set 原子操作确保同一时刻只有一个 CPU 能获取锁（kernel/spinlock.c:41），SleepLock 通过 lk->locked 标志和 sleep/wakeup 机制确保互斥（kernel/sleeplock.c:18-26）。 2. 持有并等待 (Hold-and-Wait): 成立。内核中存在持有锁后调用 sleep 的场景，如 kernel/pipe.c:63 的 pipewrite() 持有 pi->lock 后调用 sleep(&pi->nwrite, &pi->lock)；kernel/sem.c:16 的 sem_wait() 持有 sem->lock 后调用 sleep(sem, &sem->lock)。 3. 不可剥夺 (No Preemption): 成立。内核锁（SpinLock/SleepLock）不能被强制剥夺，持有锁的进程必须主动调用 release() 或 releasesleep() 释放锁。调度器不会在进程持有锁时强制剥夺其锁（仅能通过 sched() 主动让出 CPU）。 4. 循环等待 (Circular Wait): 可能成立。内核未实现全局锁顺序规范（见 Q06_016），存在嵌套锁场景（如 kernel/proc.c:583 注释提到需要父进程锁唤醒子进程），但未发现显式的 ABBA 锁检测机制。理论上可能因锁获取顺序不一致导致循环等待。	互斥：成立 (SpinLock/SleepLock 保证互斥)。持有并等待：成立 (acquiresleep 持有 lk->lk 时调用 sleep)。不可剥夺：成立 (锁不能强制释放)。循环等待：成立 (无全局锁序强制，可能产生)。
证据	[1] kernel/spinlock.c:41 · acquire while (__sync_lock_test_and_set(&lk->locked, 1) != 0) ; [2] kernel/pipe.c:63 · pipewrite acquire(&pi->lock); for (i = 0; i < n; i++) { while (pi->nwrite == pi->nread + PIPESIZE) { ... sleep(&pi->nwrite, &pi->lock); } } [3] kernel/sem.c:16 · sem_wait acquire(&sem->lock); while (sem->value <= 0) { sleep(sem, &sem->lock); } sem->value--; release(&sem->lock);	[1] kernel/spinlock.c · acquire while (__sync_lock_test_and_set(&lk->locked, 1) != 0) ; [2] kernel/sleeplock.c · acquiresleep acquire(&lk->lk); while (lk->locked) { sleep(lk, &lk->lk); } [3] kernel/proc.c · sleep Releases lk but holds p->lock while transitioning to SLEEPING.

Q06_016 · tri_state_impl

是否存在全局锁顺序（Lock Ordering）规范或注释，以预防嵌套锁导致的循环等待死锁（Circular Wait Deadlock）？（必须三态；若 implemented 需给出锁排序规则或 ABBA 锁检测代码证据）

项目	方法A	方法B
答案	stub	not_found
证据	[1] kernel/proc.c:583 · lock_ordering_comment 注释: 'we need the parent's lock in order to wake it up from wait()', 暗示需要特定锁顺序, 但未形式化规范。 [2] kernel/proc.c:821 · sleep_lock_invariant 注释: 'Must acquire p->lock in order to change p->state and then call sched', 描述 sleep 的锁不变量, 但非全局锁顺序。 [3] kernel · lock_order_grep grep 'lock_ordering lock_order ABBA' 仅命中 4 处注释, 未发现形式化的锁排序规则或 ABBA 死锁检测代码。	[1] kernel · lock ordering No explicit global lock ordering documentation or enforcement code found in the provided source.

Q06_017 · tri_state_impl

是否实现管程/条件变量 (Monitor / Condition Variable, Stallings Ch5)? (必须三态; 搜索 Condvar / condition_variable / monitor / wait/notify/signal 等; 若 implemented 需区分 Hoare 语义 (等待者立即恢复) vs Mesa 语义 (等待者重新竞争锁))

项目	方法A	方法B
答案	not_found	implemented
证据	[1] kernel · condvar_grep_scan grep 'Condvar condition_variable monitor' 仅命中 lwip 网络栈中的 SNTP_MONITOR_SERVER_REACHABILITY 宏, 内核同步模块未发现 Condvar 实现。 [2] kernel/include · sync_header_scan 检查 kernel/include/*.h (spinlock.h, sleeplock.h, sem.h, futex.h, pipe.h), 未发现 condition_variable 或 monitor 相关定义。	[1] kernel/proc.c · wakeup void wakeup(void *chan) { ... for (p = proc; p < &proc[NPROC]; p++) { ... if (p->state == SLEEPING && p->chan == chan) { p->state = RUNNABLE; } ... } } [2] kernel/sleeplock.c · releasesleep wakeup(lk); release(&lk->lk); // Wakeup called before releasing lock, but waiters must reacquire lock in acquiresleep (Mesa semantics).

Q06_018 · short_answer

经典同步问题验证 (Classic Synchronization Problems, Stallings Ch5): 以下三个经典问题在该内核中是否有对应实现或测试? - 生产者-消费者 (Producer-Consumer / Bounded Buffer): **(implemented/notfound + 证据)** - 读者-写者 (Readers-Writers): **_** (实现了读者优先/写者优先/公平? + 证据) - 哲学家就餐 (Dining Philosophers): **__** (implemented/not_found)

项目	方法A	方法B
答案	生产者 - 消费者 (Producer-Consumer / Bounded Buffer): not_found (grep 'producer.consumer bounded.buffer' 0 命中; pipe.c 的环形缓冲区实现类似有界缓冲, 但无明确的生产者 - 消费者测试用例) 读者 - 写者 (Readers-Writers): not_found (grep 'readers.writers' 0 命中; 未发现 RwLock 实现, 无读者 - 写者测试) 哲学家就餐 (Dining Philosophers): not_found (grep 'dining.philosoph' 0 命中; xv6-user/ 目录下未发现哲学家就餐测试程序)	生产者 - 消费者: implemented (Pipe 实现, kernel/pipe.c)。读者 - 写者: not_found (无 RwLock)。哲学家就餐: not_found。

项目	方法A	方法B
证据	[1] xv6-user · classic_sync_grep grep 'producer.consumer bounded.buffer readers.writers dining.philosoph ' 在 xv6-user/ 和 kernel/ 中均为 0 命中。 [2] kernel/pipe.c · pipe_bounded_buffer pipe.c 实现字节环形缓冲区（PIPESIZE=512），但未明确标注为生产者 - 消费者 模式测试。	[1] kernel/pipe.c · pipe.c Pipe implements bounded buffer with sleep/wakeup on full/empty conditions. [2] kernel/include · rwlock No Readers-Writers lock implementation found. [3] kernel · philosopher No Dining Philosophers implementation or test found.

07_security — 安全机制与权限模型

Q07_004 · short_answer

若存在权限检查：入口点与核心检查函数链路是什么？（列 2-5 个节点并给证据）

项目	方法A	方法B
答案	权限检查链路（仅 faccessat）： 1. sys_faccessat (kernel/sysfile.c:1018) - 系统调用入口 2. 检查 mode == F_OK 返回 0 (kernel/sysfile.c:1038) 3. 检查 (emode & mode) != mode 返回 -1 (kernel/sysfile.c:1041) 4. emode 硬编码为 R_OK W_OK X_OK (kernel/sysfile.c:1020) 注意：该链路未使用 proc->uid/gid 字段，仅检查传入 的 mode 参数。open/exec/write 等系统调用无权限检 查。	权限检查未实际实现。系统调用入口：sys_open/sys_exec/sys_write (kernel/sysfile.c) -> 直接调用底层函数，无 check_perm/inode_permission 等权限验证函数。UID/GID 字段仅存 在于 proc 结构中但未用于访问控制决策。
证据	[1] kernel/sysfile.c · sys_faccessat int dirfd, mode, flags, emode = R_OK W_OK X_OK; ... if (mode == F_OK) return 0; if ((emode & mode) != mode) return -1; [Source: kernel/sysfile.c:1020-1042] [2] kernel/include/file.h · R_OK #define R_OK 4, #define W_OK 2, #define X_OK 1, #define F_OK 0 [Source: kernel/include/file.h:60-63]	[1] kernel/sysfile.c · permission_check 在附件源码中检索 check_perm/inode_permission/access_check 等权限检查函数，未找到实际执行 UID/GID 验证的代码 [2] kernel/sysfile.c · open uint64 open(char *path, int omode) { ... if ((ep = ename(path)) == NULL) { return -1; } ... } - 仅检查文件是否 存在，无权限验证

Q07_007 · tri_state_impl

是否存在栈保护/溢出防护（stack canary/guard page）或等价机制？（必须三态）

项目	方法A	方法B
答案	stub	not_found

项目	方法A	方法B
证据	[1] kernel/vm.c · uvmclear uvmclear() 用于用户栈保护页: void uvmclear(pagetable_t pagetable, uint64 va) { pte = walk(pagetable, va, 0); *pte &= ~PTE_U; } [Source: kernel/vm.c:417-424] [2] kernel/include/memlayout.h · guard_page_comment 注释提及 guard page: // each surrounded by invalid guard pages. [Source: kernel/include/memlayout.h:92] [3] kernel/net.mk · stack_protector 发现 -fno-stack-protector 编译选项, 表明未启用 stack canary。 [Source: kernel/net.mk:19]	[1] Makefile · stack-protector CFLAGS += \$(shell \$(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector) - 显式禁用栈保护 [2] kernel/vm.c · guard_page 在附件源码中检索 guard page/canary/stack_guard 等关键字, 未找到栈溢出防护机制

Q07_009 · short_answer

本项目支持哪些架构 (riscv64/aarch64/x86_64/loongarch64 等)? 每种架构的安全相关初始化 (特权级配置、PMP/MPU/SMEP 等) 是否有代码证据? (必须逐架构作答, 无证据写「未发现」)

项目	方法A	方法B
答案	仅支持 RISC-V 64 架构。 RISC-V 64 安全相关初始化: 1. 特权级配置: 通过 SSTATUS_SPP 位控制用户/内核模式切换 (kernel/trap.c:144-145) 2. 中断使能: trapinithart() 设置 SSTATUS_SIE 和 SIE_* (kernel/trap.c:35-37) 3. 页表隔离: 每个进程有独立用户页表 (kernel/proc.c:256-260) 未发现 PMP (Physical Memory Protection) 配置代码。搜索 'PMP PMA SMEP SMAP' 关键词 0 命中。	仅支持 riscv64 架构。证据: 1) Makefile 中 TOOLPREFIX := riscv64-unknown-elf-; 2) kernel/include/riscv.h 定义 RISC-V 特有寄存器操作; 3) kernel/entry_visionfive.S 和 kernel/entry_qemu.S 为 RISC-V 入口代码。安全相关初始化: trapinithart() 设置 stvec/sie 寄存器启用中断, 但未发现 PMP/SMEP 等高级保护机制配置代码。
证据	[1] kernel/trap.c · usertrapret usertrapret() 设置 SPP=0 进入用户态: x &= ~SSTATUS_SPP; w_sstatus(x); [Source: kernel/trap.c:144] [2] kernel/ · grep_pmp 搜索 'PMP PMA SMEP SMAP KPTI TTBR' 关键词, 0 命中, 表明无 PMP 或等价硬件保护机制配置。[Source: kernel/ 全目录扫描] [3] kernel/entry_qemu.S · entry_qemu.S RISC-V 64 启动入口, 无架构检测或多架构支持代码。[Source: kernel/entry_qemu.S:1-19]	[1] Makefile · TOOLPREFIX TOOLPREFIX := riscv64-unknown-elf- - 仅支持 RISC-V 64 位工具链 [2] kernel/include/riscv.h · riscv.h 定义 RISC-V 特有 CSR 寄存器操作: r_sstatus/w_sstatus/r_satp/w_satp 等 [3] kernel/trap.c · trapinithart w_stvec((uint64)kernelvec); w_sstatus(r_sstatus() SSTATUS_SIE); - 设置陷阱向量和启用中断, 无 PMP 配置

Q07_012 · short_answer

UID/GID 字段是否在 syscall 路径上真实执行权限检查? (搜索 check_perm/inode_permission; 若只有字段无检查链须标注「仅有定义但未强制执行 ♦」; 给检查链证据或写「字段存在但无检查链」)

项目	方法A	方法B
答案	字段存在但无检查链  证据: 1. struct proc 有 uid/gid 字段 (kernel/include/proc.h:67-68) 2. allocproc() 初始化为 0 (kernel/proc.c:235-236) 3. sys_getuid/sys_setuid 可读写 (kernel/sysproc.c:378-390) 4. 但 sys_faccessat 未使用 uid/gid 检查 (kernel/sysfile.c:1018-1048) 5. open/exec/write 等系统调用无权限检查逻辑 6. FAT32 的 ekstat() 硬编码 st_uid/st_gid 为 0 (kernel/fat32.c:824-825) 搜索 'check_perm inode_permission' 关键词 0 命中, 无通用权限检查函数。	字段存在但无检查链。proc 结构中有 uid/gid 字段 (kernel/include/proc.h), sys_setuid/sys_setgid 可设置 (kernel/sysproc.c), 但 open/exec/write 等系统调用中未发现 check_perm/inode_permission 等权限验证函数调用。kstat 中 st_uid/st_gid 硬编码为 0 (kernel/fat32.c)。
证据	[1] kernel/include/proc.h · proc <pre>struct proc { int uid; int gid; ... }</pre> [Source: kernel/include/proc.h:67-68] [2] kernel/sysfile.c · sys_faccessat <pre>sys_faccessat 未使用 myproc()~>uid/gid, 仅检查 mode 参数。[Source: kernel/sysfile.c:1018-1048]</pre> [3] kernel/ · grep_check_perm 搜索 'check_perm inode_permission' 关键词, 0 命中, 无通用权限检查函数。[Source: kernel/ 全目录扫描]	[1] kernel/include/proc.h · proc <pre>struct proc { ... int uid; int gid; ... }; - UID/GID 字段定义</pre> [2] kernel/fat32.c · kstat <pre>kst->st_uid = 0; kst->st_gid = 0; - UID/GID 硬编码为 0</pre> [3] kernel/sysfile.c · check_perm 在附件源码中检索 check_perm/inode_permission/access_check 等权限检查函数, 在 open/write 等系统调用路径中未找到实际使用 UID/GID 进行访问控制的代码

08_network — 网络子系统与协议栈

Q08_004 · short_answer

选择一个发送路径 (优先 sys_sendto) , 追踪: syscall → 协议栈 → 网卡驱动。列 3-6 个关键节点并给证据。

项目	方法A	方法B
答案	发送路径追踪 (sys_sendto) : 1. sys_sendto (kernel/syssocket.c:254) - 系统调用入口, 解析用户参数并调用 do_sendto 2. do_sendto (kernel/socket_new.c:107) - 内核层封装, 分配内核缓冲区并调用 lwip_sendto 3. lwip_sendto (kernel/lwip/api/sockets.c:1710) - lwIP socket API, 处理 UDP/TCP 分发 4. netconn_send (kernel/lwip/api/api_lib.c) - 网络连接层发送接口 5. netif_loop_output (kernel/lwip/core/netif.c:1099) - 本地回环网络接口输出 (无真实网卡驱动) 注意: 该仓库仅实现本地回环 (loopback) , 数据通过 ring_buffer 在本地 socket 间传递, 未经过真实网卡驱动。	发送路径关键节点: 1. sys_sendto() [kernel/syssocket.c] - 系统调用入口, 获取 socket 文件描述符和参数 2. do_sendto() [kernel/lwip/api/sockets.c] - socket 层处理, 调用 lwip_sendto 3. lwip_sendto() [kernel/lwip/api/sockets.c] - lwIP API 层, 路由到 UDP/TCP 发送 4. udp_sendto()/tcp_write() [kernel/lwip/core/] - 协议层处理, 构建协议头 5. netif->output_ip6()/ethip6_output() [kernel/lwip/core/ipv6/] - 网络接口层输出 6. 底层通过 loopback 或虚拟网络接口发送 (tcpip_init_with_loopback 初始化)

项目	方法A	方法B
证据	[1] kernel/syssocket.c · sys_sendto sys_sendto() 于 kernel/syssocket.c:254 接收用户参数，调用 do_sendto(f->socketnum, buf, len, flags, ...) [2] kernel/socket_new.c · do_sendto do_sendto() 于 kernel/socket_new.c:107 分配内核缓冲区 kbuf，copyin 用户数据后调用 lwip_sendto() [3] kernel/lwip/api/sockets.c · lwip_sendto lwip_sendto() 于 kernel/lwip/api/sockets.c:1710 根据 socket 类型分发到 TCP 或 UDP 发送路径 [4] kernel/lwip/core/netif.c · netif_loop_output netif_loop_output() 于 kernel/lwip/core/netif.c:1099 处理本地回环数据包，无真实网卡驱动 [5] doc/net.md · loopback_design 文档说明：'测试程序中只存在本机回环，我们的 socket 接口采取了简化的实现方法，不经过 qemu 的网卡，直接通过本机 ring buffer 进行信息传递'	[1] kernel/syssocket.c · sys_sendto 系统调用入口，调用 do_sendto(f->socketnum, buf, len, flags, ...) [2] kernel/lwip/api/sockets.c · lwip_sendto socket API 层，根据 socket 类型调用 udp_sendto 或 tcp_write [3] kernel/lwip/core/udp.c · udp_sendto UDP 协议层发送函数，构建 UDP 头并调用 ip_output [4] kernel/lwip/core/ipv6/ethip6.c · ethip6_output 以太网 IPv6 输出函数，处理链路层封装 [5] kernel/lwip/api/tcpip.c · tcpip_init_with_loopback 初始化 lwIP 协议栈，使用 loopback 模式进行络通信

Q08_006 · multi_choice

协议支持情况（多选；未发现则留空并在 notes 写 not_found）：

项目	方法A	方法B
答案	["B. ARP", "C. IPv4/IPv6", "E. UDP", "F. TCP", "H. DNS"]	["Ethernet", "ARP", "IPv4/IPv6", "ICMP", "UDP", "TCP", "DHCP", "DNS"]
证据	[1] kernel/lwip/lwipopts.h · lwip_config LWIP_ARP=1 (ARP 支持)、LWIP_IPV4=1 (IPv4 支持)、LWIP_UDP=1 (UDP 支持)、LWIP_TCP=1 (TCP 支持)、LWIP_DNS=1 (DNS 支持) [2] kernel/lwip/core/tcp.c · tcp_impl TCP 协议实现于 kernel/lwip/core/tcp.c (2639 行)，包含 tcp_connect、tcp_send 等核心函数 [3] kernel/lwip/core/udp.c · udp_impl UDP 协议实现于 kernel/lwip/core/udp.c (1331 行)，包含 udp_send、udp_recv 等核心函数 [4] kernel/lwip/include/lwip/sockets.h · socket_types 定义 SOCK_STREAM=1 (TCP)、SOCK_DGRAM=2 (UDP)，证实 TCP/UDP socket 类型支持	[1] kernel/lwip/include/lwip/opt.h · opt.h lwIP 配置文件，定义 LWIP_ETHERNET, LWIP_ARP, LWIP_IPV4, LWIP_IPV6, LWIP_ICMP, LWIP_ICMP6, LWIP_UDP, LWIP_TCP, LWIP_DHCP, LWIP_DHCP6, LWIP_DNS 等协议支持选项 [2] kernel/lwip/core/etharp.c · etharp.c ARP 协议实现文件 [3] kernel/lwip/core/ipv4/ip4.c · ip4.c IPv4 协议实现 [4] kernel/lwip/core/ipv6/ip6.c · ip6.c IPv6 协议实现 [5] kernel/lwip/core/icmp.c · icmp.c ICMPv4 协议实现 [6] kernel/lwip/core/ipv6/icmp6.c · icmp6.c ICMPv6 协议实现 [7] kernel/lwip/core/udp.c · udp.c UDP 协议实现 [8] kernel/lwip/core/tcp.c · tcp.c TCP 协议实现 [9] kernel/lwip/core/ipv4/dhcp.c · dhcp.c DHCPv4 客户端实现 [10] kernel/lwip/core/ipv6/dhcp6.c · dhcp6.c DHCPv6 客户端实现 [11] kernel/lwip/core/dns.c · dns.c DNS 解析器实现

09_debug_error — 调试机制与错误处理

Q09_003 · short_answer

panic 路径会输出哪些诊断？（寄存器 dump/栈回溯/停机；必须引用实现证据）

项目	方法A	方法B
答案	panic 路径输出：1) 错误字符串 (serious_print 输出 s); 2) 调用 backtrace() 输出栈回溯 (kernel/printf.c:280-289); 3) 内核态 panic 额外输出 scause/sepc/stval/hartid/pid/name (kernel/trap.c:176-180); 4) 用户态未处理异常输出 scause/sepc/ir 并调用 trapframedump 输出全部寄存器 (kernel/trap.c:94-99, kernel/trap.c:241-274); 5) 设置 panicked=1 后进入无限循环停机 (kernel/printf.c:275-277)。	Panic message, Stack trace (via backtrace), Halt (infinite loop). Register dump is not explicitly in panic() but backtrace prints return addresses.
证据	<pre>[1] kernel/printf.c:266-277 · panic void panic(char s) { serious_print(s); backtrace(); panicked=1; for(;;); } [2] kernel/printf.c:280-289 · backtrace backtrace 基于帧指针遍历: while(fp<bottom) { ra= (fp-1); serious_print(ra-4); fp=*(fp-2); } [3] kernel/trap.c:241-274 · trapframedump trapframedump 输出 trapframe 中全部寄存器: a0-a7/t0-t6/s0-s11/ra/sp/gp/tp/epc [4] kernel/trap.c:176-180 · kerneltrap serious_print("\nscause %p\n", scause); serious_print("sepc=%p stval=%p hart=%d\n"); panic("kerneltrap");</pre>	<pre>[1] kernel/printf.c · panic void panic(char s) { ... serious_print("panic: "); serious_print(s); serious_print("\n"); backtrace(); panicked = 1; for (;;) ; } [2] kernel/printf.c · backtrace void backtrace() { ... while (fp < bottom) { uint64 ra = (fp - 1); serious_print("%p\n", ra - 4); fp = (uint64)(fp - 2); } }</pre>

Q09_007 · short_answer

错误码/错误类型体系是什么？（errno/Result/Error enum；给类型定义与传播点证据）

项目	方法A	方法B
答案	errno 风格错误码体系：1) kernel/include/error.h:4-12 定义 enum ErrorCode { UNKNOWN_ERROR=1, BAD_PROCESS, INVALID_PARAM, NO_FREE_MEMORY, NO_FREE_PROCESS, NOT_ELF_FILE, INVALID_PROCESS_STATUS, INVALID_PERM }; 2) kernel/include/error.h:14-121 定义标准 POSIX errno 宏 (EPERM=1, ENOENT=2, ESRCH=3, EINTR=4, EIO=5, ENOMEM=12, EACCES=13, EINVAL=22 等共 121 个); 3) 系统调用通过 return -1 或 return -E* 传播错误 (如 kernel/sysfile.c:1286 sys_syslog 返回 -1); 4) 内核函数通过 return 错误码或直接调用 panic 处理致命错误。	Kernel uses global 'errno' (int) defined in errno.h. lwIP subsystem uses 'err_t' (enum). Syscalls return -1 on error.

项目	方法A	方法B
证据	[1] kernel/include/error.h:4-12 · ErrorCode enum ErrorCode { UNKNOWN_ERROR=1, BAD_PROCESS, INVALID_PARAM, NO_FREE_MEMORY, NO_FREE_PROCESS, NOT_ELF_FILE, INVALID_PROCESS_STATUS, INVALID_PERM } [2] kernel/include/error.h:14-60 · errno_macros #define EPERM 1, #define ENOENT 2, #define ENOMEM 12, #define EACCES 13, #define EINVAL 22 等标准 errno 宏 [3] kernel/sysfile.c:1286 · sys_syslog if (argint(0,&type)<0 argaddr(1,&bufp)<0 argint(2,&len)<0) return -1; 错误返回 -1 [4] kernel/proc.c:545 · exit exit(-1) 用于进程异常终止，传递错误状态给用户态	[1] kernel/include/errno.h · errno.h #define EPERM 1 ... extern int errno; [2] kernel/lwip/include/lwip/err.h · err.h typedef enum { ERR_OK = 0, ERR_MEM = -1, ... } err_enum_t; typedef s8_t err_t; [3] kernel/sysfile.c · sys_open if (argstr(0, path, FAT32_MAX_PATH) < 0 argint(1, &omode) < 0) return -1;

Q09_008 · tri_state_impl

是否存在 trace/perf/fttrace 等跟踪机制或 tracepoints？（必须三态）

项目	方法A	方法B
答案	not_found	implemented
证据	[1] kernel/ · tracepoint_scan 全局搜索 tracepoint fttrace perf_event trace_ 关键字, 0 命中（已搜索 416 个文件） [2] kernel/printf.c:72-125 · debug_print debug_print 为条件编译的调试日志宏（#ifdef DEBUG），非 tracepoint 机制，无动态启用/禁用或事件过滤功能 [3] kernel/trap.c:51-100 · usertrap usertrap 处理异常但无性能计数/事件采样逻辑，无 perf_event_open 或类似系统调用	[1] kernel/syscall.c · sys_trace uint64 sys_trace(void) { int mask; if (argint(0, &mask) < 0) return -1; myproc()->tmask = mask; return 0; } [2] kernel/syscall.c · syscall if ((p->tmask & (1 << num)) != 0) { printf("pid %d: %s -> %d\n", p->pid, sysnames[num], p->trapframe->a0); }